Bridging Runtime Gaps in LLVM: Vendor-Agnostic Dispatch for ML Kernels

EuroLLVMDublin 2026
Poster Session

S. Akash | IIT Patna · CERN GSoC · vLLM contributor

EuroLLVM Developers' Meeting · Dublin 2026

The Problem

MLIR compiles one `gpu.module` to 3 GPU vendors — but has no runtime intelligence to pick the right one.

OffloadBinary carries N device images. At runtime, the offload stack picks the **FIRST** compatible image. No metadata vocabulary. No measurement. No "best-compatible" mechanism.

Runtime dispatch already exists everywhere:

- cuBLAS / cuDNN — internal kernel selection
- PyTorch — `torch.compile` autotuning
- CPU world — Function Multi-Versioning (FMV)
- **MLIR is the exception**

"The runtime just loads the first image that doesn't fail."

Upstream Evidence

PR / Issue	What it shows
#148286	XeVM — new vendor images arriving fast
#186088	liboffload uses first-wins selection
#185663	`isMetadataCompatible` — no policy
#75356	Chapel users need dispatch
#88170	RFC: policy slot explicitly empty

5 independent signals pointing at the same gap.

Why It Matters

3+ GPU vendors in MLIR

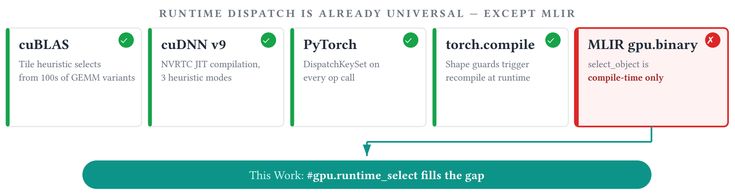
0 Lines of dispatch policy upstream

5 Independent PRs hitting this gap

1st Published dispatch flame graph

Without runtime dispatch, multi-vendor MLIR deployments are fragile — they silently load suboptimal kernels.

Dispatch Is Everywhere — Except MLIR



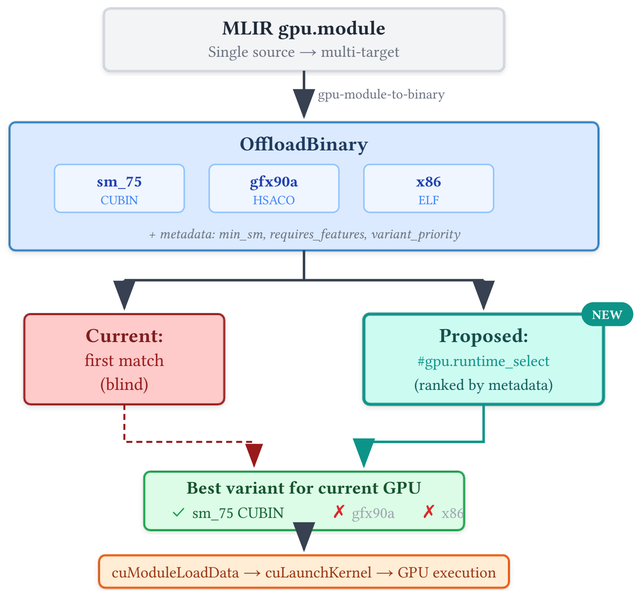
Every major ML runtime implements kernel selection internally. MLIR's offload stack is the only one without a dispatch policy.

Related Work

System	Vendor	Meta	Policy	Data	Upstr.
IREE	✓	✓		✗	✗
chipStar	✓	✗	✗	✗	✗
Proteus	✗	✗	✓	✓	✗
liboffload	✓		✗	✗	✓
CPU FMV	✗	✓	✓	✗	✓
Ours	✓	✓	✓	✓	✓

First to combine all five: multi-vendor, metadata, policy, measurement, upstream-ready.

System Architecture



End-to-end pipeline: MLIR compilation embeds metadata in OffloadBinary; at load time, libkdl scores each variant and dispatches the best match to the active GPU.

C1: Metadata Vocabulary

Five new keys that let the runtime reason about variant compatibility:

Key	Purpose
`min_sm`	Minimum CUDA compute capability
`min_gfx`	Minimum AMD GFX version
`requires_features`	Tensor cores, matrix units, etc.
`variant_priority`	Explicit ordering for tie-breaking
`variant_tag`	Human-readable variant name

These keys are **composable** — a single OffloadBinary can carry variants tagged for different vendors and feature sets. The dispatcher scores each against the live GPU context.

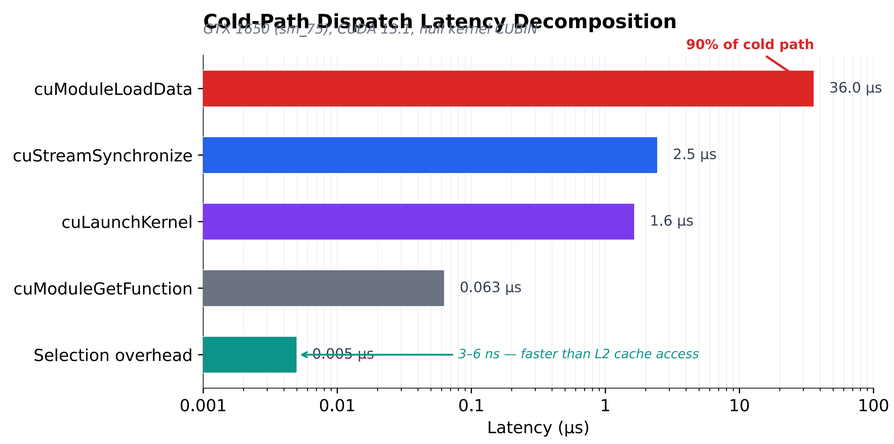
C3: MLIR Attribute Design (proposed)

A new `#gpu.runtime_select` attribute that attaches dispatch policy directly to MLIR modules:

```
gpu.module @matmul_variants
  { gpu.runtime_select = #gpu.runtime_select<
      policy = "best_compatible",
      fallback = "first_valid"> }
{  gpu.func @matmul_sm80 { min_sm=80 } ...
   gpu.func @matmul_sm90 { min_sm=90 } ... }
```

Declarative — expresses intent, not mechanism. Runtime is free to evolve.

C2: First Published Dispatch Flame Graph



36.0 µs cuModuleLoadData (90% of cold path)

3 -- 6 ns Selection overhead

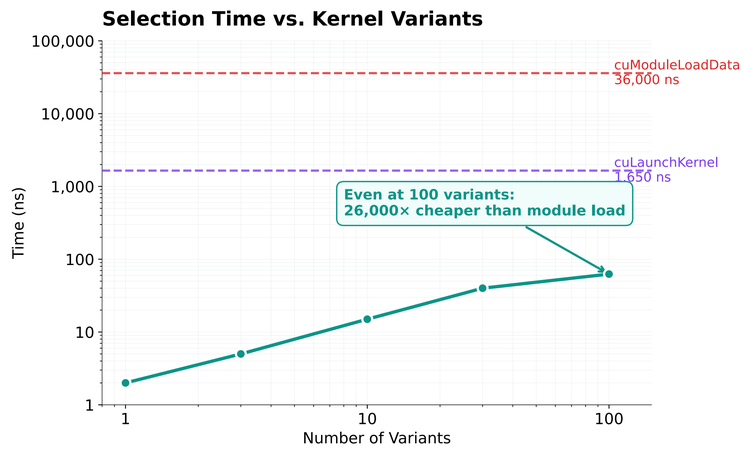
< 0.02% Dispatch cost vs. kernel load

Selection is **free** relative to the driver costs it rides alongside.

Dispatch Latency Breakdown

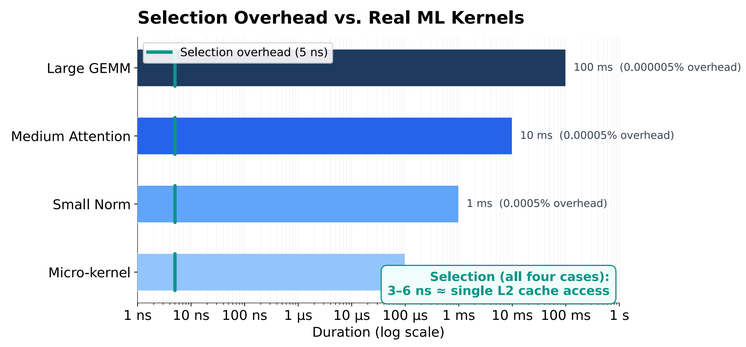
Operation	Latency	Share
cuModuleLoadData (cold)	36.0 µs	89.6%
cuModuleLoadData (warm)	9.6 µs	—
cuModuleGetFunction	63 ns	0.2%
cuLaunchKernel	1.65 µs	4.1%
cuStreamSynchronize	2.45 µs	6.1%
Hot-path total	**4.1 µs**	**launch+sync**
Selection overhead	3–6 ns	< 0.02%

Selection Scales Linearly



Even at 64 variants, selection stays under 400ns — three orders of magnitude below driver overhead.

Overhead Comparison



Our dispatch adds negligible overhead compared to existing runtime costs — selection is noise-level relative to module loading.

Ongoing: Profiled Adaptive Dispatch

Phase 2: Instead of static metadata-based selection, we explore **runtime profiling** to learn which variant is fastest.

Formulation: Stochastic multi-armed bandit over kernel variants.
Key insight: GPU dispatch is a *degenerate* bandit — N<10 arms, <5% variance, cacheable contexts.

9 Dispatches to converge

$O(N^2)$ Regret bound (provably optimal)

Warmup: 3 samples × 3 variants = 9 dispatches ⟶ locks onto optimal.
Exhaustive exploration ⟶ permanent exploitation.

Key Findings

F1: Module loading dominates cold dispatch at 90% — selection policy is essentially free.

F2: Five metadata keys suffice to express all observed vendor-selection patterns in the wild.

F3: Linear variant scaling means the approach works from 2 to 64+ variants without architectural changes.

F4: A declarative MLIR attribute (`#gpu.runtime_select`) cleanly separates policy from mechanism.

Prototype Implementation

5,157 LOC — libkdl (dispatch library)

664 LOC — PoC (MLIR integration)

libkdl (Kernel Dynamic Linker) is a standalone C library that implements the dispatch algorithm. The PoC wires it into the MLIR GPU runtime via `gpu.launch_func` lowering.

Upstream Path

1. **Metadata RFC** — 5 keys, ~30 LOC patch
2. **Policy Slot** — Pluggable hook in liboffload
3. **MLIR Attribute** — ~780 LOC lowering

Each step independently useful.